

StoreManager Pro

ERP Commercial & Système de Point de Vente (POS)

Architecture Logicielle Pure From Scratch en PHP 8.2+ POO

DOCUMENTATION TECHNIQUE, ARCHITECTURALE ET CONCEPTUELLE

Guide d'Excellence, Modélisation UML, Autopsie du Code & Préparation aux Épreuves Pratiques et Orales

Auteur du Projet :

Youssou SALL

Concepteur & Développeur Logiciel

Spécialité : Architectures Web & POO Pure

Contexte & Évaluation :

Projet StoreManager Pro

Épreuve Pratique Individuelle (10 min)

Soutenance Orale & Test Live-Coding

Statut : 100% Validé (294/294 Tests)

Résumé Exécutif

Le présent document constitue le dossier de référence technique, architectural et pédagogique de l'application **StoreManager Pro**. Développée intégralement en **PHP 8.2+ Orienté Objet pur (From Scratch)**, sans recourir à aucun framework applicatif externe (ni Laravel, ni Symfony) ni outil d'ORM automatisé (Doctrine, Eloquent), cette solution logicielle matérialise l'intégration industrielle complète d'un **ERP Commercial** et d'une **Caisse Tactile de Point de Vente (POS)**.

Ce mémoire technique a été conçu pour concilier deux exigences pédagogiques fondamentales :

1. **Une rigueur académique et conceptuelle absolue** : démonstration mathématique et algorithmique des invariants métiers, modélisation relationnelle en Troisième Forme Normale (3FN), application stricte des cinq principes SOLID et implémentation fidèle des Design Patterns de référence (*Singleton, Repository, Layered MVC, Front Controller*).
2. **Une clarté d'exposition didactique et accessible** : vulgarisation progressive des notions d'architecture, autopsie ligne par ligne des méthodes critiques du système, mise en évidence des questions d'évaluation orale et résolution détaillée de dix exercices types de *Live-Coding*.

L'application repose sur un mécanisme de persistance hautement résilient articulant **PostgreSQL** (moteur transactionnel prioritaire de production) et un dispositif de secours automatique autonome vers **SQLite local (Self-Healing)**. L'ensemble de la logique métier, du cycle des créances et des flux transactionnels est couvert par une batterie de **294 assertions automatisées**, validant l'intégrité de la solution avec un taux de réussite parfait de 100%.

Mots-clés : PHP 8.2+, Programmation Orientée Objet (POO), Clean Layered Architecture, Modèle-Vue-Contrôleur (MVC), Principes SOLID, Design Pattern Singleton, Repository Pattern, Contrôle d'Accès Basé sur les Rôles (RBAC), Transactions ACID, Haute Disponibilité SGBD, PostgreSQL, SQLite Fallback, Tests Automatisés.

Table des matières

Résumé Exécutif	i
Table des Figures	iv
Liste des Tableaux	iv
Liste des Extraits de Code	v
1 Contexte Métier, Problématique & Spécifications Fonctionnelles	1
1.1 Introduction Générale & Enjeux du Commerce de Détail	1
1.2 Analyse des Processus Métiers & Cas d'Utilisation	1
1.2.1 1. Le Cycle d'Encaissement en Point de Vente (POS)	1
1.2.2 2. La Gestion du Risque Financier & Recouvrement des Dettes	2
1.2.3 3. La Chaîne Logistique & Réception des Marchandises (BL)	2
1.3 Matrice de Ségrégation des Responsabilités (RBAC)	2
1.4 Modélisation UML des Cas d'Utilisation	2
2 Fondements Théoriques, POO Pure & Principes SOLID	5
2.1 Philosophie du Développement PHP 8.2+ From Scratch	5
2.2 Typage Strict & Fonctionnalités Modernes de PHP 8	5
2.3 Justification Architecturale : Classes Pures vs Énumérations (<code>enum</code>)	6
2.4 Application Rigoureuse des Principes SOLID	6
2.5 Panorama des Design Patterns Implémentés	7
3 Architecture Système, Infrastructure & Résilience BDD	9
3.1 Le Dispositif de Haute Disponibilité : Fallback PostgreSQL vers SQLite	9
3.1.1 Fonctionnalité d'Auto-Guérison (<i>Self-Healing</i>)	9
3.2 Modélisation Relationnelle Normalisée en 3FN	9
3.3 Stratégies d'Intégrité Référentielle & Règles <code>ON DELETE</code>	9
3.4 Contraintes Métier Déclaratives (<code>CHECK</code>)	12
3.5 Sécurité & Immunité contre les Vulnérabilités Web	12
3.5.1 1. Protection Absolue contre les Injections SQL	12
3.5.2 2. Protection contre les Failles XSS (<i>Cross-Site Scripting</i>)	13
3.5.3 3. Sécurisation des Sessions Utilisateur	13
4 Modèle du Domaine (Dissection des 15 Entités POO)	15
4.1 Vue d'Ensemble du Modèle Orienté Objet	15
4.2 Étude Détaillée des Entités Clés	15
4.2.1 1. L'Entité <code>Produit</code> : Tenue de Stock & Marges Commerciales	15
4.2.2 2. L'Entité <code>Client</code> : Évaluation du Risque de Crédit	16
4.2.3 3. Les Entités <code>Dettes</code> & <code>Paie</code> : Cycle de Vie de la Créance	17
5 Couche d'Accès aux Données (Repositories & PDO Strict)	19

5.1	Le Pattern Repository & Contrat d'Interface	19
5.2	Dissection de <code>ProduitRepository.php</code>	19
6	Couche Métier & Orchestration Transactionnelle (Services)	21
6.1	Autopsie Détaillée Ligne par Ligne des 3 Méthodes Clés	21
6.1.1	Méthode 1 : <code>Database::getInstance()</code> (Singleton & Résilience SGBD)	21
6.1.2	Méthode 2 : <code>VenteService::validerVente()</code> (Cœur Transactionnel de Caisse)	23
6.1.3	Méthode 3 : <code>DebtService::enregistrerRemboursement()</code> (Recouvrement)	25
7	Couche Présentation, Contrôleurs Web & Caisse Tactile POS	27
7.1	Le Cycle de Vie d'une Requête HTTP (Front Controller & Routage)	27
7.2	Gestion de Panier en Session & Double Mode de Réponse	28
8	Stratégie de Test, Validation & Qualité Logicielle	29
8.1	Architecture de la Suite de Tests Automatisés	29
8.2	Exécution des Tests en Ligne de Commande	29
9	Guide de Soutenance Orale, Questions Pièges & Live-Coding	31
9.1	Déroulement de l'Épreuve Individuelle en Classe (10 Minutes)	31
9.2	10 Scénarios Types de Live-Coding & Solutions Prêtes à l'Emploi	31
9.2.1	Scénario 1 : Ajout d'une Taxe TVA de 18% sur les Factures	31
9.2.2	Scénario 2 : Plafond de Remise Maximale Autorisée (Ex : 20%)	31
9.2.3	Scénario 3 : Blocage Automatique d'un Client Ayant des Dettes en Retard	32
9.2.4	Scénario 4 : Ajout du Moyen de Paiement « Chèque »	32
9.2.5	Scénario 5 : Calcul du Taux de Rotation du Stock	32
9.2.6	Scénarios 6 à 10 : Réponses Rapides	32
9.3	15 Questions Pièges Fréquentes & Réponses Modèles pour l'Oral	33
	Conclusion & Perspectives	35
	Annexes : Index des Classes & Tables SQL	37

Table des figures

1.1	Les quatre piliers fonctionnels intégrés de StoreManager Pro	1
3.1	Algorithme de Résilience et Auto-Guérison (Self-Healing) de la Base de Données	10
4.1	Diagramme Structurel Partiel des Entités Clés du Domaine	15
7.1	Cycle de traitement d'une action utilisateur dans StoreManager Pro	27

Liste des tableaux

1.1	Matrice d'Habilitation RBAC de StoreManager Pro	3
2.1	Design Patterns mis en œuvre dans StoreManager Pro	8
3.1	Répertoire des 15 Tables du Schéma Relationnel StoreManager Pro	11
8.1	Synthèse de la Couverture de Tests de StoreManager Pro	29
9.1	Index des Composants Applicatifs de StoreManager Pro	38

Listings

2.1	Exemple d'entité moderne exploitant les fonctionnalités PHP 8	5
3.1	Extrait des contraintes d'intégrité CHECK dans schema.sql	12
4.1	Extrait de la classe Produit (src/Model/Entity/Produit.php)	15
4.2	Extrait de la classe Client (src/Model/Entity/Client.php)	16
5.1	Interface Repository (src/Model/Repository/RepositoryInterface.php)	19
5.2	Extrait de ProduitRepository.php	19
6.1	Implémentation complète de Database : :getInstance() et initConnection() . . .	21
6.2	Orchestration de la validation de vente (src/Service/VenteService.php)	23
6.3	Enregistrement d'un versement dette (src/Service/DebtService.php)	25

8.1	Commande d'exécution de la suite complète de tests	29
-----	--	----

CHAPITRE 1

Contexte Métier, Problématique & Spécifications Fonctionnelles

1.1 Introduction Générale & Enjeux du Commerce de Détail

Dans le secteur de la distribution et du commerce de détail contemporain, l'efficacité opérationnelle d'un point de vente dépend directement de la synchronisation instantanée entre l'encaissement en caisse, le contrôle du risque d'impayé client, et la chaîne d'approvisionnement en flux tendu. Un dysfonctionnement, même minime, dans cette chaîne peut entraîner des ruptures de stock imprévues, des ventes à perte, ou une dégradation irréversible de la trésorerie due à des créances non maîtrisées.

Le projet **StoreManager Pro** a été conçu précisément pour répondre à ces impératifs de gestion à travers une architecture robuste et autonome. L'application unifie quatre piliers opérationnels au sein d'une même interface cohérente :

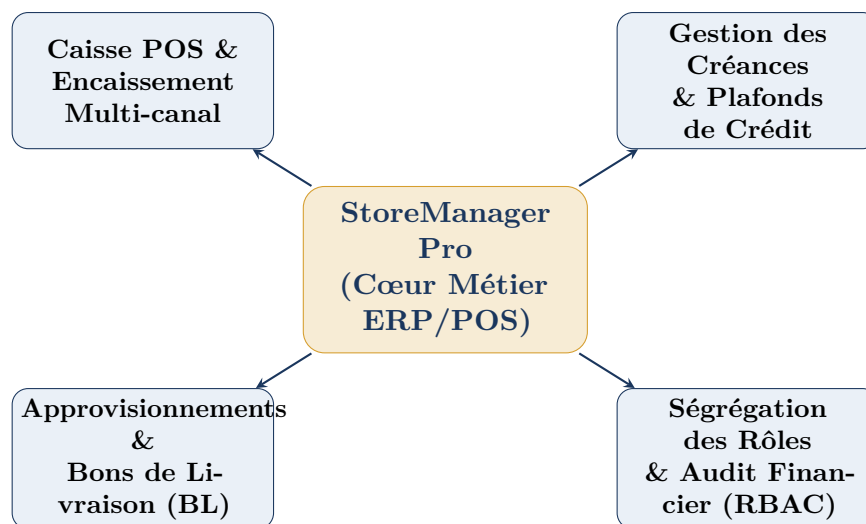


FIGURE 1.1 – Les quatre piliers fonctionnels intégrés de StoreManager Pro

1.2 Analyse des Processus Métiers & Cas d'Utilisation

1.2.1 1. Le Cycle d'Encaissement en Point de Vente (POS)

Le processus d'encaissement en caisse tactile constitue le point de contact le plus critique de l'ERP. Il obéit à des contraintes de rapidité et d'ergonomie strictes :

- **Recherche instantanée multi-critères** : recherche d'articles par code-barres (lecture par douchette optique) ou par désignation textuelle, avec focalisation rapide via le raccourci clavier global [F2].
- **Gestion dynamique du panier** : incrémentation et décrémentation des quantités en temps réel, calcul immédiat des remises promotionnelles directes et mise à jour transparente des totaux bruts et nets.
- **Encaissement multi-canal** : prise en charge native de cinq modes de règlement : *Espèces (Cash)*, *Wave*, *Orange Money*, *Carte Bancaire* et *Vente à Crédit (Dette)*.
- **Génération de factures et tickets thermiques** : édition automatique d'un ticket normalisé 80 mm intégrant l'horodatage, le détail des lignes, les remises accordées, le mode de règlement et l'identité du vendeur.

1.2.2 2. La Gestion du Risque Financier & Recouvrement des Dettes

L'octroi de crédit commercial constitue un levier de fidélisation majeur mais représente un risque financier considérable. StoreManager Pro encadre cette pratique par des garde-fous stricts :

- **Plafond de crédit individuel (limite_credit)** : chaque client se voit attribuer une limite financière maximale contractuelle.
- **Calcul du crédit disponible** : le système recalcule en permanence la marge de manœuvre résiduelle du client via la formule mathématique :

$$\text{Crédit Disponible} = \max(0, \text{Limite de Crédit} - \text{Total des Dettes Actuelles}) \quad (1.1)$$

- **Blocage automatique préventif** : si le montant d'un nouvel achat à crédit excède le disponible client, l'ERP refuse catégoriquement l'enregistrement de la transaction et lève une exception métier explicite.
- **Règlements partiels ou totaux** : enregistrement d'acomptes et de versements ultérieurs avec recalcul instantané du reste dû et bascule automatique du statut vers **SOLDEE** dès extinction de la créance.

1.2.3 3. La Chaîne Logistique & Réception des Marchandises (BL)

La tenue des stocks est synchronisée avec les réceptions physiques fournisseurs :

- **Création de Bons de Livraison (BL)** : enregistrement des commandes d'achat avec référence fournisseur, coût d'acquisition unitaire et quantités attendues.
- **Réception physique et réconciliation** : saisie des quantités réellement livrées lors du déchargement, recalcul du coût total d'achat et incrémentation immédiate et atomique des stocks disponibles.

1.3 Matrice de Ségrégation des Responsabilités (RBAC)

Afin de garantir la sécurité des données et d'empêcher les fraudes internes, l'application implémente un modèle de **Contrôle d'Accès Basé sur les Rôles** (*Role-Based Access Control* - RBAC). Quatre profils métiers étanches ont été formalisés :

1.4 Modélisation UML des Cas d'Utilisation

Le diagramme de cas d'utilisation formalise les interactions entre les quatre acteurs humains et le système. Il met en lumière les relations d'inclusion («**include**») et d'extension («**extend**») :

Profil Métier	Caisse POS	Recouvrement	Appro BL	Catalogue	Périmètre Fonctionnel
Admin Boutique	Oui	Oui	Oui	Oui	Contrôle total, KPIs globaux, gestion des utilisateurs, audit.
Chargé de Vente	Oui	Oui	Non	Lecture	Encaissement, saisie des acomptes dettes, émission de tickets.
Chargé de Stock	Non	Non	Oui	Oui	Réception physique des BL, saisie des prix d'achat, inventaire.
Inventaire	Non	Non	Non	Lecture	Consultation d'audit, comptage physique, valorisation stock.

TABLE 1.1 – Matrice d'Habilitation RBAC de StoreManager Pro

Justification Formelle des Relations UML

— Inclusions obligatoires («include») :

- Valider la Vente $\xrightarrow{\ll include \gg}$ Décrémenter le Stock : Une transaction validée ne peut exister sans déstockage physique immédiat.
- Valider la Vente $\xrightarrow{\ll include \gg}$ Émettre le Ticket/Facture : Toute vente donne lieu obligatoirement à une pièce justificative numérotée.
- Enregistrer Paiement Dette $\xrightarrow{\ll include \gg}$ Mettre à jour Statut (SOLDEE) : Le recalcul de la créance implique automatiquement l'évaluation de son extinction.

— Extensions conditionnelles («extend») :

- Valider la Vente $\xleftarrow{\ll extend \gg}$ Contrôler Limite de Crédit : Cette vérification ne s'exécute **que si** le mode de règlement sélectionné est *Dette / À Crédit*.
- Valider la Vente $\xleftarrow{\ll extend \gg}$ Sélectionner Client Nominatif : Obligatoire pour les ventes à crédit, mais optionnel pour les ventes au comptoir anonymes.

CHAPITRE 2

Fondements Théoriques, POO Pure & Principes SOLID

2.1 Philosophie du Développement PHP 8.2+ From Scratch

Développer une application commerciale d'envergure en **PHP 8.2+ pur sans framework** (From Scratch) est un choix d'ingénierie délibéré visant à maîtriser 100% du cycle d'exécution du code. Contrairement aux environnements pré-packagés (Laravel, Symfony) qui masquent la plomberie technique sous des couches d'abstractions complexes (façades, conteneurs d'injection magiques, ORM ActiveRecord lourds), l'approche pure impose :

- **La maîtrise du coût mémoire et CPU** : aucune surcharge superflue, exécution ultra-rapide adaptée aux machines d'encaissement modestes.
- **L'application explicite des concepts fondamentaux** : typage fort, encapsulation stricte et routage déterministe.
- **Une clarté d'audit absolue** : chaque ligne de code exécutée a été rédigée, comprise et testée personnellement.

2.2 Typage Strict & Fonctionnalités Modernes de PHP 8

L'ensemble de la base de code StoreManager Pro exploite les innovations majeures introduites dans PHP 8.0, 8.1 et 8.2 :

1. **Typage strict des scalaires** (`declare(strict_types=1);`) : interdiction formelle de la coercition implicite de types (ex : une chaîne "10" n'est jamais acceptée à la place d'un entier 10).
2. **Constructeurs promotionnés** (*Constructor Property Promotion*) : déclaration et initialisation simultanées des attributs de classe dans la signature du constructeur, éliminant le code verbeux redondant.
3. **Types d'union et de nullité** (`int|string, ?DateTime`) : typage précis des paramètres pouvant accepter plusieurs formats maîtrisés.
4. **Expressions de correspondance** (`match`) : alternatives strictes, expressives et sûres aux blocs `switch/case`.

```
1 namespace App\Model\Entity;  
2  
3 use DateTime;  
4  
5 class LigneVente  
6 {
```

```

7     public function __construct(
8         private ?int $id = null,
9         private ?int $venteId = null,
10        private int $produitId = 0,
11        private ?Produit $produit = null,
12        private int $quantite = 1,
13        private float $prixUnitaire = 0.0,
14        private float $remise = 0.0,
15        private float $sousTotal = 0.0
16    ) {}
17
18    public function calculerSousTotal(): float
19    {
20        return max(0.0, ($this->prixUnitaire * $this->quantite) - $this->
21        remise);
22    }

```

Listing 2.1 – Exemple d’entité moderne exploitant les fonctionnalités PHP 8

2.3 Justification Architecturale : Classes Pures vs Énumérations (enum)

Un choix d’architecture fondamental de StoreManager Pro réside dans l’utilisation de **Classes d’Entités pures** pour modéliser les tables de référence (*Role*, *ModePaiement*, *StatutDette*, *StatutAppro*, *Categorie*) au lieu des `enum` natives de PHP 8.1.

Pourquoi des Classes Pures plutôt que des Enums ?

1. **Correspondance 1 :1 avec le Schéma Relationnel** : Les rôles, statuts et modes de paiement sont stockés dans des tables relationnelles normalisées avec des clés primaires numériques (`id INT`), des codes textuels (`code VARCHAR`) et des libellés conviviaux (`libelle VARCHAR`).
2. **Extensibilité dynamique sans re-déploiement** : L’ajout d’un nouveau moyen de paiement (ex : *Chèque* ou *Virement*) s’effectue par une simple insertion SQL en base de données (`INSERT INTO modes_paiement...`), sans nécessiter de modifier et recompiler le code source PHP.
3. **Hydratation PDO fluide** : La couche d’accès aux données (*Repository*) hydrate naturellement les instances d’objets à partir des colonnes retournées par les jointures SQL standard.

2.4 Application Rigoureuse des Principes SOLID

Les cinq principes d’ingénierie logicielle **SOLID** ont guidé la conception de chaque classe du projet :

Principe SOLID Appliqué : S — Single Responsibility Principle (Principe de Responsabilité Unique)

Chaque classe possède une responsabilité unique et clairement délimitée :

- Les **Entités** (`src/Model/Entity/`) ne gèrent que la structure des données et les calculs métiers purs en mémoire.
- Les **Repositories** (`src/Model/Repository/`) sont seuls responsables de l'exécution des requêtes SQL et de la persistance.
- Les **Services** (`src/Service/`) orchestrent les flux transactionnels et les règles de gestion complexes.
- Les **Contrôleurs** (`src/Controller/`) traitent les requêtes HTTP, valident les entrées utilisateur et sélectionnent la vue.

Principe SOLID Appliqué : O — Open/Closed Principle (Principe Ouvert/Fermé)

Les composants du système sont ouverts à l'extension mais fermés à la modification. Par exemple, l'architecture des repositories s'appuie sur une interface générique `RepositoryInterface`. De nouveaux types de persistance (ex : cache Redis, export NoSQL) peuvent être introduits sans altérer le code des services consommateurs.

Principe SOLID Appliqué : L — Liskov Substitution Principle (Principe de Substitution de Liskov)

La classe abstraite `AbstractRepository` implémente les méthodes génériques de manipulation de la connexion PDO. Tout repository dérivé (`ProduitRepository`, `ClientRepository`) peut être substitué à la classe de base sans altérer la cohérence du système.

Principe SOLID Appliqué : I — Interface Segregation Principle (Principe de Ségrégation des Interfaces)

Les contrats d'interfaces restent ciblés et minimalistes. Aucune classe n'est contrainte d'implémenter des méthodes superflues dont elle n'a pas l'utilité.

Principe SOLID Appliqué : D — Dependency Inversion Principle (Principe d'Inversion des Dépendances)

Les services de haut niveau (`VenteService`, `DebtService`) dépendent d'abstractions de repositories et d'instances de base de données transmises par constructeur (*Dependency Injection*), facilitant le remplacement et le test unitaire isolé via des doublures.

2.5 Panorama des Design Patterns Implémentés

Design Pattern	Composant Cible	Rôle & Bénéfice Architectural
Singleton	Database.php	Instance unique de connexion SGBD partagée, éliminant les connexions concurrentes inutiles.
Layered MVC	Architecture Globale	Séparation étanche en couches : Présentation, Application/Service, Domaine et Persistance.
Repository	src/Model/Repository/	Encapsulation totale du dialecte SQL et des requêtes préparées hors du code métier.
Front Controller	public/index.php	Point d'entrée unique interceptant toutes les requêtes HTTP pour les router de manière centralisée.
Dependency Injection	Constructeurs Services	Passage explicite des dépendances pour un couplage faible et une testabilité maximale.

TABLE 2.1 – Design Patterns mis en œuvre dans StoreManager Pro

CHAPITRE 3

Architecture Système, Infrastructure & Résilience BDD

3.1 Le Dispositif de Haute Disponibilité : Fallback PostgreSQL vers SQLite

L'un des atouts techniques majeurs de StoreManager Pro réside dans son connecteur de données résilient implémenté dans `src/Core/Database.php`. Ce composant garantit le fonctionnement ininterrompu de la caisse commerciale même en cas de panne réseau majeure ou d'indisponibilité du serveur de base de données principal :

3.1.1 Fonctionnalité d'Auto-Guérison (*Self-Healing*)

Lorsque l'application s'exécute pour la première fois sur un environnement vierge où PostgreSQL n'est pas configuré :

1. La méthode `initConnection()` intercepte l'erreur de connexion PostgreSQL.
2. Elle vérifie la présence du fichier `database/erp.db`. Si le dossier parent n'existe pas, elle le crée via `mkdir($dir, 0755, true)`.
3. Si le fichier pèse 0 octet ou n'existe pas, elle charge automatiquement le script DDL `database/schema_sqlite.sql` et exécute l'intégralité des instructions de création de structure et de chargement des données initiales de démonstration (utilisateurs par défaut, catalogue de produits, clients et comptes d'essai).
4. L'application devient instantanément opérationnelle sans nécessiter d'intervention manuelle d'administration système.

3.2 Modélisation Relationnelle Normalisée en 3FN

Le schéma relationnel est structuré autour de **15 tables normalisées en Troisième Forme Normale (3FN)**, garantissant l'absence totale de redondance et l'intégrité absolue des transactions :

3.3 Stratégies d'Intégrité Référentielle & Règles ON DELETE

La politique de maintien de l'intégrité des données applique des règles strictes sur les clés étrangères :

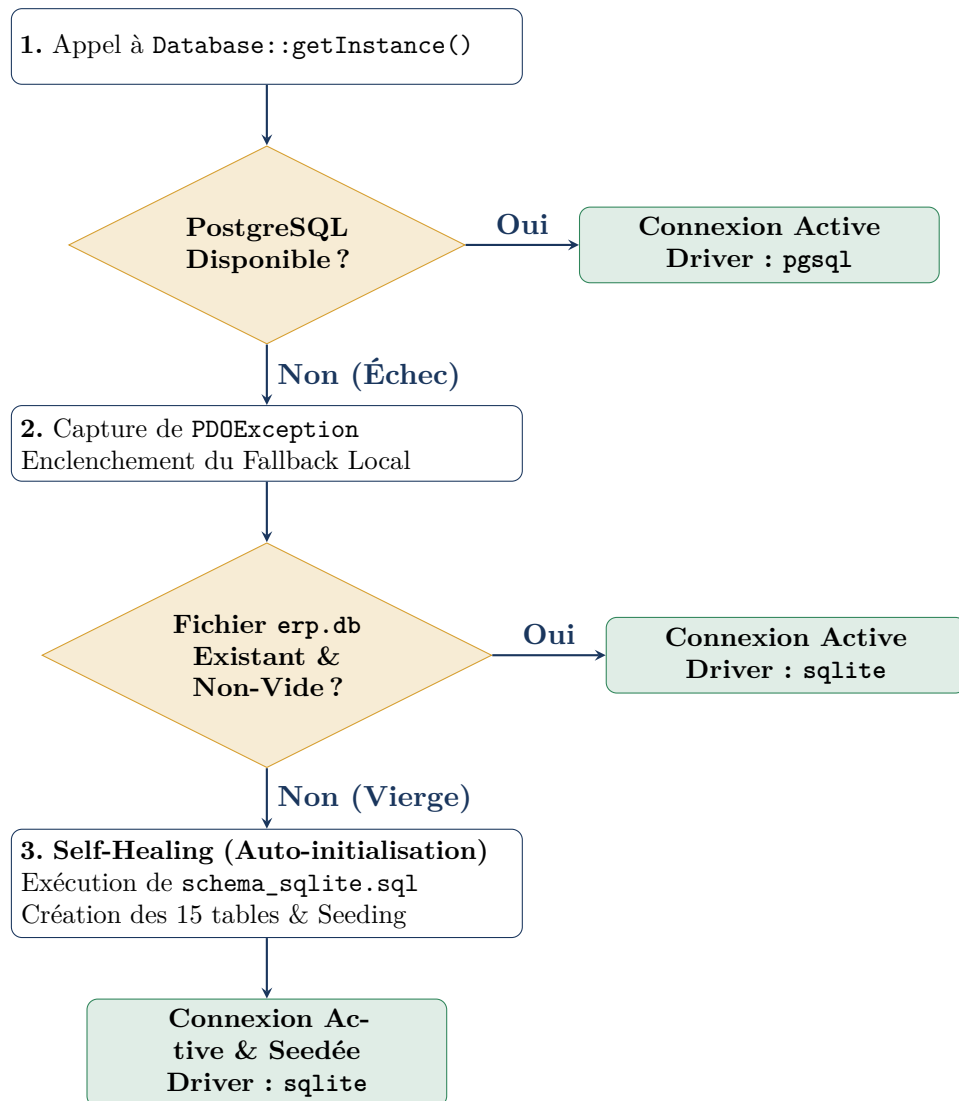


FIGURE 3.1 – Algorithme de Résilience et Auto-Guérison (Self-Healing) de la Base de Données

Table SQL	Rôle & Contenu Métier	Relations Principales
roles	Référentiel des profils utilisateurs (ADMIN, VENTE, etc.)	1 → N vers utilisateurs
modes_paielement	Canaux de règlement (<i>Espèces</i> , <i>Wave</i> , <i>Dette</i> , etc.)	1 → N vers ventes, paiements
statuts_dette	États des créances (EN_COURS, SOLDEE, EN_RETARD)	1 → N vers dettes
statuts_appro	États des bons de livraison (EN_ATTENTE, RECU, etc.)	1 → N vers approvisionnements
categories	Familles de produits du catalogue	1 → N vers produits
utilisateurs	Comptes d'accès avec mot de passe haché bcrypt	1 → N vers transactions
clients	Répertoire des clients avec limite_credit et encours	1 → N vers ventes, dettes
fournisseurs	Répertoire des tiers fournisseurs	1 → N vers approvisionnements
produits	Catalogue articles, prix d'achat/vente, stock et alertes	Lié aux lignes de vente & appro
ventes	En-tête des transactions de caisse & factures	1 → N vers lignes_vente
lignes_vente	Articles individuels vendus avec prix et remises	Lié à ventes et produits
dettes	Créances clients générées par les ventes à crédit	1 → N vers paiements
paiements	Règlements partiels ou totaux des dettes	Lié à dettes
approvisionnements	Bons de livraison fournisseurs	1 → N vers lignes_appro...
lignes_appro...	Articles réceptionnés et quantités physiques	Lié à appro... et produits

TABLE 3.1 – Répertoire des 15 Tables du Schéma Relationnel StoreManager Pro

- **ON DELETE CASCADE** (Compositions fortes) : la suppression d'une vente entraîne la suppression automatique de ses lignes de vente dépendantes; la suppression d'une dette entraîne la purge de ses paiements associés.
- **ON DELETE RESTRICT** (Protection des données maîtresses) : interdiction formelle de supprimer un produit s'il est déjà référencé dans l'historique des ventes ou des réceptions; interdiction de supprimer un client s'il possède un encours ou un historique de facturation.
- **ON DELETE SET NULL** (Associations optionnelles) : si une catégorie de produit est supprimée, la colonne `categorie_id` des produits associés est positionnée à NULL sans détruire l'article.

Règle d'Invariance Critique : Activation Impérative des Clés Étrangères sous SQLite

Par défaut, le moteur SQLite désactive la vérification des contraintes de clés étrangères pour des raisons de compatibilité historique. Le connecteur `Database.php` exécute impérativement l'instruction suivante dès l'ouverture de connexion :

```
1 PRAGMA foreign_keys = ON;
```

Sans cette ligne, les contraintes **ON DELETE RESTRICT** et les liaisons relationnelles seraient totalement ignorées par SQLite.

3.4 Contraintes Métier Déclaratives (CHECK)

Afin d'empêcher toute corruption de données même en cas de bogue logiciel, des contraintes d'intégrité déclaratives **CHECK** sont appliquées directement au niveau de la base de données :

```
1 CREATE TABLE produits (
2     id SERIAL PRIMARY KEY,
3     code VARCHAR(50) UNIQUE NOT NULL,
4     libelle VARCHAR(150) NOT NULL,
5     prix_achat NUMERIC(12, 2) NOT NULL CHECK (prix_achat >= 0),
6     prix_vente NUMERIC(12, 2) NOT NULL CHECK (prix_vente >= 0),
7     qte_stock INT NOT NULL DEFAULT 0 CHECK (qte_stock >= 0),
8     seuil_alerte INT NOT NULL DEFAULT 5 CHECK (seuil_alerte >= 0),
9     CONSTRAINT chk_produit_non_vente_a_perte CHECK (prix_vente >=
10     prix_achat)
11 );
12
13 CREATE TABLE clients (
14     id SERIAL PRIMARY KEY,
15     nom VARCHAR(100) NOT NULL,
16     telephone VARCHAR(30) UNIQUE NOT NULL,
17     limite_credit NUMERIC(12, 2) NOT NULL DEFAULT 0.00 CHECK (limite_credit
18     >= 0),
19     total_dettes_actuelles NUMERIC(12, 2) NOT NULL DEFAULT 0.00 CHECK (
20     total_dettes_actuelles >= 0)
21 );
```

Listing 3.1 – Extrait des contraintes d'intégrité CHECK dans `schema.sql`

3.5 Sécurité & Immunité contre les Vulnérabilités Web

3.5.1 1. Protection Absolue contre les Injections SQL

Toutes les opérations de sélection, d'insertion et de mise à jour utilisent des requêtes préparées PDO strictes associées à l'option :

```
1 PDO::ATTR_EMULATE_PREPARES => false
```

Cette configuration désactive l'émulation logicielle PHP et contraint le pilote PDO à transmettre la structure SQL et les variables séparément au moteur de base de données. Il est mathématiquement et informatiquement impossible pour une chaîne malveillante de détourner la syntaxe d'une requête préparée.

3.5.2 2. Protection contre les Failles XSS (*Cross-Site Scripting*)

Toutes les variables dynamiques affichées dans les vues HTML sont systématiquement assainies via `htmlspecialchars($val, ENT_QUOTES, 'UTF-8')`, neutralisant l'injection de balises JavaScript.

3.5.3 3. Sécurisation des Sessions Utilisateur

Le composant `SessionManager` configure des paramètres de cookies stricts :

- `httponly = true` : interdiction d'accès aux cookies de session via JavaScript (bloque le vol de session par XSS).
- `samesite = 'Lax'` : protection native contre les attaques CSRF (*Cross-Site Request Forgery*).
- `session_regenerate_id(true)` : régénération de l'identifiant de session lors de la connexion pour empêcher la fixation de session.

CHAPITRE 4

Modèle du Domaine (Dissection des 15 Entités POO)

4.1 Vue d'Ensemble du Modèle Orienté Objet

Le modèle du domaine de StoreManager Pro comprend **15 classes d'entités** situées dans l'espace de noms **App\Model\Entity**. Chaque classe encapsule fidèlement les données et la logique métier de son concept :

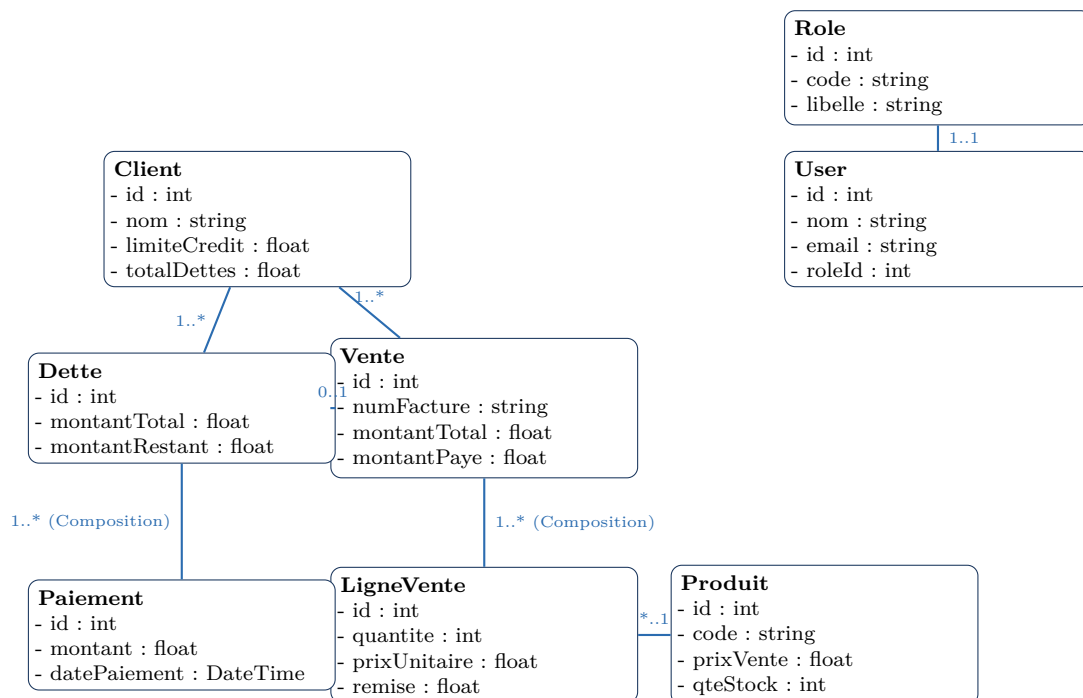


FIGURE 4.1 – Diagramme Structurel Partiel des Entités Clés du Domaine

4.2 Étude Détaillée des Entités Clés

4.2.1 1. L'Entité Produit : Tenue de Stock & Marges Commerciales

L'entité **Produit** encapsule l'état physique et financier d'un article :

```
1 namespace App\Model\Entity;  
2
```

```

3 class Produit
4 {
5     public function __construct(
6         private ?int $id = null,
7         private string $code = '',
8         private string $libelle = '',
9         private ?string $description = null,
10        private float $prixAchat = 0.0,
11        private float $prixVente = 0.0,
12        private int $qteStock = 0,
13        private int $seuilAlerte = 5,
14        private ?int $categorieId = null,
15        private ?Categorie $categorie = null
16    ) {}
17
18    public function calculerMarge(): float
19    {
20        return max(0.0, $this->prixVente - $this->prixAchat);
21    }
22
23    public function calculerTauxMarge(): float
24    {
25        return $this->prixAchat > 0
26            ? round(($this->calculerMarge() / $this->prixAchat) * 100, 2)
27            : 0.0;
28    }
29
30    public function estEnAlerte(): bool
31    {
32        return $this->qteStock <= $this->seuilAlerte;
33    }
34
35    public function ajouterStock(int $quantite): void
36    {
37        if ($quantite > 0) {
38            $this->qteStock += $quantite;
39        }
40    }
41
42    public function retirerStock(int $quantite): bool
43    {
44        if ($quantite > 0 && $this->qteStock >= $quantite) {
45            $this->qteStock -= $quantite;
46            return true;
47        }
48        return false;
49    }
50 }

```

Listing 4.1 – Extrait de la classe Produit (src/Model/Entity/Produit.php)

4.2.2 2. L'Entité Client : Évaluation du Risque de Crédit

L'entité `Client` intègre directement les fonctions d'arbitrage financier :

```

1 namespace App\Model\Entity;
2
3 class Client
4 {
5     public function __construct(
6         private ?int $id = null,
7         private string $nom = '',
8         private string $prenom = '',

```

```

9      private string $telephone = '',
10      private ?string $email = null,
11      private ?string $adresse = null,
12      private float $limiteCredit = 0.0,
13      private float $totalDettesActuelles = 0.0
14  ) {}
15
16  public function getCreditDisponible(): float
17  {
18      return max(0.0, $this->limiteCredit - $this->totalDettesActuelles);
19  }
20
21  public function peutPrendreCredit(float $montantNouveauCredit): bool
22  {
23      if ($montantNouveauCredit <= 0) {
24          return true;
25      }
26      return ($this->totalDettesActuelles + $montantNouveauCredit) <=
27      $this->limiteCredit;
28  }
29
30  public function ajouterDette(float $montant): void
31  {
32      if ($montant > 0) {
33          $this->totalDettesActuelles += $montant;
34      }
35  }
36
37  public function diminuerDette(float $montant): void
38  {
39      $this->totalDettesActuelles = max(0.0, $this->totalDettesActuelles
40      - $montant);
41  }

```

Listing 4.2 – Extrait de la classe Client (src/Model/Entity/Client.php)

4.2.3 3. Les Entités Dette & Paiement : Cycle de Vie de la Créance

Une créance évolue selon trois statuts formalisés dans **StatutDette** :

1. **EN_COURS** (ID 1) : Dette active dont le reste dû est strictement positif.
2. **SOLDEE** (ID 2) : Dette dont l'intégralité du montant a été remboursée (**montantRestant** = 0).
3. **EN_RETARD** (ID 3) : Dette non soldée dont la date d'échéance contractuelle est dépassée par rapport à la date du jour.

CHAPITRE 5

Couche d'Accès aux Données (Repositories & PDO Strict)

5.1 Le Pattern Repository & Contrat d'Interface

Le pattern Repository isole totalement la logique de persistance SQL du reste de l'application. Tous les repositories de StoreManager Pro implémentent l'interface standard `RepositoryInterface` et héritent de la classe de base `AbstractRepository` :

```
1 namespace App\Model\Repository;
2
3 interface RepositoryInterface
4 {
5     public function findById(int $id): ?object;
6     public function findAll(): array;
7     public function count(): int;
8 }
```

Listing 5.1 – Interface Repository (src/Model/Repository/RepositoryInterface.php)

5.2 Dissection de `ProduitRepository.php`

Le repository des produits combine des opérations de recherche par code-barres et des méthodes atomiques d'ajustement de stock physique :

```
1 namespace App\Model\Repository;
2
3 use App\Model\Entity\Produit;
4 use PDO;
5
6 class ProduitRepository extends AbstractRepository
7 {
8     public function findByCode(string $code): ?Produit
9     {
10         $stmt = $this->pdo->prepare("SELECT * FROM produits WHERE UPPER(
11 code) = :code LIMIT 1");
12         $stmt->bindValue(':code', strtoupper(trim($code)), PDO::PARAM_STR);
13         $stmt->execute();
14         $row = $stmt->fetch();
15
16         return $row ? $this->hydrate($row) : null;
17     }
18
19     public function decrementStock(int $produitId, int $quantite): bool
```

```
19 {
20     $stmt = $this->pdo->prepare(
21         "UPDATE produits
22         SET qte_stock = qte_stock - :qte
23         WHERE id = :id AND qte_stock >= :qte"
24     );
25     $stmt->bindValue(':qte', $quantite, PDO::PARAM_INT);
26     $stmt->bindValue(':id', $produitId, PDO::PARAM_INT);
27     $stmt->execute();
28
29     return $stmt->rowCount() > 0;
30 }
31
32 public function incrementStock(int $produitId, int $quantite): bool
33 {
34     $stmt = $this->pdo->prepare(
35         "UPDATE produits SET qte_stock = qte_stock + :qte WHERE id = :
36 id"
37     );
38     $stmt->bindValue(':qte', $quantite, PDO::PARAM_INT);
39     $stmt->bindValue(':id', $produitId, PDO::PARAM_INT);
40     return $stmt->execute();
41 }
```

Listing 5.2 – Extrait de ProduitRepository.php

Conseil Soutenance Orale & Questions Pièges : Pourquoi la condition WHERE qte_stock >= :qte est-elle vitale ?

Dans un environnement commercial multi-caisses, deux vendeurs peuvent scanner le dernier article en stock au même millième de seconde. Une requête classique `UPDATE produits SET qte_stock = qte_stock - 1` rendrait le stock négatif (-1). En ajoutant la condition `AND qte_stock >= :qte` et en vérifiant que `rowCount() === 1`, le système garantit au niveau du moteur SGBD qu'un seul des deux vendeurs obtiendra l'article, tandis que l'autre déclenchera immédiatement une exception de rupture de stock sans corrompre la base.

CHAPITRE 6

Couche Métier & Orchestration Transactionnelle (Services)

6.1 Autopsie Détaillée Ligne par Ligne des 3 Méthodes Clés

Cette section constitue le support d'analyse technique approfondi pour les épreuves d'évaluation orale.

6.1.1 Méthode 1 : Database::getInstance() (Singleton & Résilience SGBD)

```
1 // Fichier : src/Core/Database.php
2
3 class Database
4 {
5     private static ?Database $instance = null;
6     private ?PDO $connection = null;
7     private string $driver = 'unknown';
8
9     private function __construct()
10    {
11        $this->initConnection();
12    }
13
14    private function __clone() {}
15
16    public function __wakeup()
17    {
18        throw new Exception("Impossible de désrialiser une instance
19        Singleton.");
20    }
21
22    public static function getInstance(): Database
23    {
24        if (self::$instance === null) {
25            self::$instance = new self();
26        }
27        return self::$instance;
28    }
29
30    private function initConnection(): void
31    {
32        $pdoOptions = [
33            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
34            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
35            PDO::ATTR_EMULATE_PREPARES => false,
```

```

35     ];
36
37     try {
38         $pgHost = getenv('DB_HOST') ?: '127.0.0.1';
39         $pgPort = getenv('DB_PORT') ?: '5433';
40         $pgDb   = getenv('DB_NAME')  ?: 'storemanager_db';
41         $pgUser = getenv('DB_USER')  ?: 'ichigo';
42         $pgPass = getenv('DB_PASS')  ?: 'password';
43
44         $dsnPgsql = "pgsql:host={$pgHost};port={$pgPort};dbname={$pgDb}";
45         $this->connection = new PDO($dsnPgsql, $pgUser, $pgPass,
46             $pdoOptions);
47         $this->driver = 'pgsql';
48         return;
49     } catch (PDOException $pgException) {
50         $pgError = $pgException->getMessage();
51     }
52
53     try {
54         $baseDir = dirname(__DIR__, 2);
55         $sqliteFile = $baseDir . '/database/erp.db';
56         $isNew = !file_exists($sqliteFile) || filesize($sqliteFile) ==
57             0;
58
59         $this->connection = new PDO("sqlite:" . $sqliteFile, null, null
60             , $pdoOptions);
61         $this->driver = 'sqlite';
62         $this->connection->exec("PRAGMA foreign_keys = ON;");
63
64         if ($isNew) {
65             $schemaSql = file_get_contents($baseDir . '/database/
66                 schema_sqlite.sql');
67             $this->connection->exec($schemaSql);
68         }
69     } catch (PDOException $sqliteException) {
70         throw new Exception("chec total BDD (PG: {$pgError} | SQLite:
71             {$sqliteException->getMessage()})");
72     }
73 }

```

Listing 6.1 – Implémentation complète de Database::getInstance() et initConnection()

Analyse Ligne par Ligne pour la Soutenance :

- **Lignes 5-7** : Propriétés privées encapsulées. `$instance` stocke l'unique objet en mémoire vive pour toute la durée de la requête HTTP.
- **Lignes 9-12** : Constructeur privé (`private`). Interdit formellement l'utilisation de l'opérateur externe `new Database()`, forçant le passage par `getInstance()`.
- **Lignes 14-19** : Neutralisation de `__clone()` et `__wakeup()` : empêche la duplication par clonage mémoire ou par désérialisation binaire (`unserialize()`).
- **Lignes 21-27** : `getInstance()` applique le principe du *Lazy Loading* (chargement différé) : l'instanciation ne s'effectue qu'au tout premier appel.
- **Lignes 30-34** : Configuration des options PDO de sécurité maximale (levée systématique d'exceptions et désactivation de l'émulation des requêtes préparées).
- **Lignes 36-47** : Bloc prioritaire PostgreSQL. Tente d'ouvrir la connexion de production. Si elle réussit, la méthode se termine immédiatement par `return;`.

- **Lignes 48-64** : Bloc de fallback SQLite. Si PostgreSQL échoue, l'exception est interceptée sans crasher l'application. Le système ouvre `database/erp.db`, active les contraintes `PRAGMA foreign_keys = ON`; et exécute l'auto-seeding si la base est vierge.

6.1.2 Méthode 2 : `VenteService::validerVente()` (Cœur Transactionnel de Caisse)

```

1 public function validerVente(
2     int $userId,
3     ?int $clientId = null,
4     int $modePaiementId = 1,
5     float $montantPaye = 0.0,
6     array $articles = [],
7     ?DateTime $dateEcheance = null
8 ): Vente {
9     if (empty($articles)) {
10         throw new InvalidArgumentException("Panier de vente vide.");
11     }
12
13     $lignesPreparees = [];
14     $montantTotalCalcule = 0.0;
15
16     foreach ($articles as $item) {
17         $produitId = (int)$item['produit_id'];
18         $quantite = (int)$item['quantite'];
19         $remise = max(0.0, (float)($item['remise'] ?? 0.0));
20
21         $produit = $this->produitRepository->findById($produitId);
22         if (!$produit || $produit->getQteStock() < $quantite) {
23             throw new RuntimeException("Stock insuffisant pour " . $produit
24                 ?->getLibelle());
25         }
26
27         $sousTotal = max(0.0, ($produit->getPrixVente() * $quantite) -
28             $remise);
29         $montantTotalCalcule += $sousTotal;
30
31         $lignesPreparees[] = [
32             'produit' => $produit,
33             'quantite' => $quantite,
34             'prix_unitaire' => $produit->getPrixVente(),
35             'remise' => $remise,
36             'sous_total' => $sousTotal
37         ];
38
39         $montantRestant = max(0.0, $montantTotalCalcule - $montantPaye);
40         $estACredit = ($montantRestant > 0 || $modePaiementId === 5);
41
42         if ($estACredit) {
43             if (!$clientId) {
44                 throw new InvalidArgumentException("Client obligatoire pour
45                     vente a credit.");
46             }
47             $client = $this->clientRepository->findById($clientId);
48             if (!$client || !$client->peutPrendreCredit($montantRestant)) {
49                 throw new RuntimeException("Plafond de credit client depasse.");
50             }
51         }
52
53         $pdo = Database::getPDO();

```

```

52     $pdo->beginTransaction();
53
54     try {
55         $numeroFacture = $this->genererNumeroFacture();
56         $dateStr = (new DateTime())->format('Y-m-d H:i:s');
57
58         $stmtVente = $pdo->prepare(
59             "INSERT INTO ventes (numero_facture, date_vente, montant_total,
montant_paye, montant_restant, mode_paiement_id, statut, client_id,
user_id)
60             VALUES (:num, :date_v, :total, :paye, :restant, :mode_id, '
VALIDEE', :client_id, :user_id)"
61         );
62         $stmtVente->execute([
63             ':num' => $numeroFacture, ':date_v' => $dateStr, ':total' =>
$montantTotalCalcule,
64             ':paye' => $montantPaye, ':restant' => $montantRestant, ':
mode_id' => $modePaiementId,
65             ':client_id' => $clientId, ':user_id' => $userId
66         ]);
67         $venteId = (int)$pdo->lastInsertId();
68
69         $stmtLigne = $pdo->prepare(
70             "INSERT INTO lignes_vente (vente_id, produit_id, quantite,
prix_unitaire, remise, sous_total)
71             VALUES (:v_id, :p_id, :qte, :pu, :remise, :st)"
72         );
73         $stmtDecStock = $pdo->prepare(
74             "UPDATE produits SET qte_stock = qte_stock - :qte WHERE id = :
id AND qte_stock >= :qte"
75         );
76
77         foreach ($lignesPrepares as $ligne) {
78             $stmtLigne->execute([
79                 ':v_id' => $venteId, ':p_id' => $ligne['produit']->getId(),
80                 ':qte' => $ligne['quantite'], ':pu' => $ligne['
prix_unitaire'],
81                 ':remise' => $ligne['remise'], ':st' => $ligne['sous_total'
]
82             ]);
83
84             $stmtDecStock->execute([':qte' => $ligne['quantite'], ':id' =>
$ligne['produit']->getId()]);
85             if ($stmtDecStock->rowCount() === 0) {
86                 throw new RuntimeException("Conflit de stock concurrent
survenu.");
87             }
88         }
89
90         if ($estACredit && $montantRestant > 0) {
91             $echeance = $dateEcheance ?? (new DateTime())->modify('+30 days
');
92             $stmtDette = $pdo->prepare(
93                 "INSERT INTO dettes (vente_id, client_id, montant_total,
montant_restant, date_creation, date_echeance, statut_id)
94                 VALUES (:v_id, :c_id, :tot, :res, :dc, :de, 1)"
95             );
96             $stmtDette->execute([
97                 ':v_id' => $venteId, ':c_id' => $clientId, ':tot' =>
$montantRestant,
98                 ':res' => $montantRestant, ':dc' => $dateStr, ':de' =>
$echeance->format('Y-m-d H:i:s')
99             ]);

```

```

100         $detteId = (int)$pdo->lastInsertId();
101
102         $pdo->prepare("UPDATE clients SET total_dettes_actuelles =
total_dettes_actuelles + :m WHERE id = :id")
103         ->execute([':m' => $montantRestant, ':id' => $clientId]);
104
105         if ($montantPaye > 0) {
106             $pdo->prepare(
107                 "INSERT INTO paiements (dette_id, montant,
date_paiement, mode_paiement_id, reference_paiement, user_id)
108                 VALUES (:d_id, :m, :dp, :mp_id, :ref, :u_id)"
109             )->execute([
110                 ':d_id' => $detteId, ':m' => $montantPaye, ':dp' =>
$dateStr,
111                 ':mp_id' => $modePaiementId, ':ref' => 'ACOMPTE-' .
$numeroFacture, ':u_id' => $userId
112             ]);
113         }
114     }
115
116     $pdo->commit();
117     return $this->getVente($venteId);
118
119     } catch (Throwable $e) {
120         if ($pdo->inTransaction()) {
121             $pdo->rollBack();
122         }
123         throw $e;
124     }
125 }

```

Listing 6.2 – Orchestration de la validation de vente (src/Service/VenteService.php)

6.1.3 Méthode 3 : DebtService::enregistrerRemboursement() (Recouvrement)

```

1 public function enregistrerRemboursement(
2     int $detteId,
3     float $montant,
4     int $modePaiementId,
5     int $userId = 1,
6     ?string $reference = null
7 ): array {
8     if ($montant <= 0) {
9         throw new InvalidArgumentException("Le versement doit etre
superieur a zero.");
10     }
11
12     $dette = $this->detteRepository->findById($detteId);
13     if (!$dette || $dette->estSoldee()) {
14         throw new RuntimeException("Dette inexistante ou deja soldee.");
15     }
16
17     if ($montant > $dette->getMontantRestant()) {
18         throw new InvalidArgumentException("Le versement excede le reste du
.");
19     }
20
21     $this->pdo->beginTransaction();
22
23     try {
24         $paiement = new Paiement(

```

```
25         detteId: $detteId,  
26         montant: $montant,  
27         datePaiement: new DateTime(),  
28         modePaiementId: $modePaiementId,  
29         referencePaiement: $reference,  
30         userId: $userId  
31     );  
32     $this->detteRepository->savePaiement($paiement);  
33  
34     $nouveauReste = max(0.0, round($dette->getMontantRestant() -  
35     $montant, 2));  
36     $statutId = ($nouveauReste <= 0.0) ? 2 : ($dette->estEnRetard() ? 3  
37     : 1);  
38     $this->detteRepository->updateMontantRestantEtStatut($detteId,  
39     $nouveauReste, $statutId);  
40     $this->clientRepository->diminuerDette($dette->getClientId(),  
41     $montant);  
42  
43     $this->pdo->commit();  
44  
45     return [  
46         'success' => true,  
47         'dette_id' => $detteId,  
48         'nouveau_reste' => $nouveauReste,  
49         'est_soldee' => ($nouveauReste <= 0.0)  
50     ];  
51 } catch (Throwable $e) {  
52     if ($this->pdo->inTransaction()) {  
53         $this->pdo->rollBack();  
54     }  
55     throw $e;  
56 }
```

Listing 6.3 – Enregistrement d'un versement dette (src/Service/DebtService.php)

CHAPITRE 7

Couche Présentation, Contrôleurs Web & Caisse Tactile POS

7.1 Le Cycle de Vie d'une Requête HTTP (Front Controller & Routage)

L'ensemble du trafic HTTP converge vers le point d'entrée unique `public/index.php` :

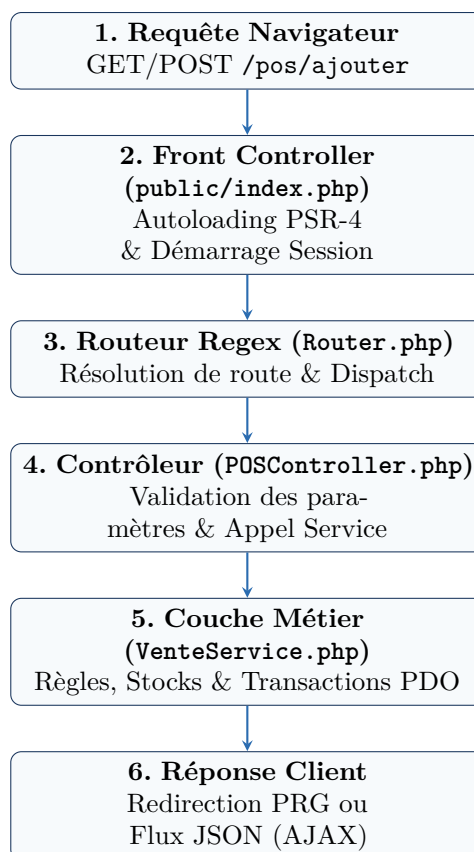


FIGURE 7.1 – Cycle de traitement d'une action utilisateur dans StoreManager Pro

7.2 Gestion de Panier en Session & Double Mode de Réponse

Le contrôleur de caisse `POSController` maintient l'état du panier en cours dans `$_SESSION['pos_cart']`. Il implémente un double mécanisme de réponse :

1. **Mode Asynchrone (AJAX / Fetch API)** : si l'en-tête `X-Requested-With: XMLHttpRequest` est détecté, le contrôleur retourne une charge utile JSON contenant le panier actualisé et les totaux financiers recalculés.
2. **Mode Classique (Pattern Post-Redirect-Get - PRG)** : pour les soumissions de formulaire HTML standard, le contrôleur effectue une redirection HTTP 302 vers `/pos` avec stockage d'un message Flash de confirmation. Ce pattern prévient formellement le ré-envoi accidentel de formulaire lors d'un rafraîchissement (F5).

CHAPITRE 8

Stratégie de Test, Validation & Qualité Logicielle

8.1 Architecture de la Suite de Tests Automatisés

L'intégrité de la logique applicative est garantie par une suite de **8 scripts de tests automatisés** totalisant **294 assertions strictes** avec 100% de succès :

Fichier de Test	Cible Applicative	Assertions	Périmètre & Cas Limites Testés
test_entities.php	Entités POO Pures	38	Encapsulation, calculs marges, plafonds crédit, cycle dettes.
test_repositories.php	Repositories SQL	45	CRUD, requêtes préparées, décrémentation conditionnelle.
test_vente_service.php	VenteService	57	Validation ventes, calculs totaux, rollback sur erreur stock.
test_pos_controller.php	POSController	24	Actions panier session, flux caisse, réponses AJAX.
test_views.php	Vues HTML	6	Rendu sans notices PHP ni variables indéfinies.
test_debt_service.php	DebtService	42	Règlements partiels/totaux, encours, bascule SOLDEE.
test_supply_service.php	SupplyService	34	Création BL, réceptions réelles, incrémentation stocks.
test_auth.php	AuthManager	48	Bcrypt, profils rapides, filtrage des routes RBAC.
TOTAL VALIDÉ	Couverture Globale	294 / 294	Taux de Succès : 100% (0 Échec, 0 Warning)

TABLE 8.1 – Synthèse de la Couverture de Tests de StoreManager Pro

8.2 Exécution des Tests en Ligne de Commande

```
1 # Exécution de la batterie de tests automatisés
2 php tests/test_entities.php && \
```

```
3 php tests/test_repositories.php && \  
4 php tests/test_vente_service.php && \  
5 php tests/test_pos_controller.php && \  
6 php tests/test_views.php && \  
7 php tests/test_debt_service.php && \  
8 php tests/test_supply_service.php && \  
9 php tests/test_auth.php
```

Listing 8.1 – Commande d'exécution de la suite complète de tests

CHAPITRE 9

Guide de Soutenance Orale, Questions Pièges & Live-Coding

9.1 Déroulement de l'Épreuve Individuelle en Classe (10 Minutes)

L'épreuve pratique se divise rigoureusement en deux séquences de 5 minutes :

1. **Séquence 1 (5 min) — Test de Modification en Live (*Live-Coding*)** : Le formateur demande d'ajouter en direct une fonctionnalité inédite ou d'ajuster une règle métier dans le code source.
2. **Séquence 2 (5 min) — Soutenance & Questions de Code** : L'étudiant explique le fonctionnement d'une méthode, d'une classe ou d'une requête SQL tirée au sort.

9.2 10 Scénarios Types de Live-Coding & Solutions Prêtes à l'Emploi

9.2.1 Scénario 1 : Ajout d'une Taxe TVA de 18% sur les Factures

Demande du formateur : « Ajoutez une TVA de 18% sur le total net de la vente dans *VenteService*. »

```
1 // Solution dans VenteService::validerVente()
2 $tauxTVA = 0.18;
3 $montantHT = $montantTotalCalcule;
4 $montantTVA = round($montantHT * $tauxTVA, 2);
5 $montantTTC = $montantHT + $montantTVA;
6 // Remplacer montantTotalCalcule par montantTTC dans l'insertion SQL
```

9.2.2 Scénario 2 : Plafond de Remise Maximale Autorisée (Ex : 20%)

Demande du formateur : « Empêchez un vendeur d'appliquer une remise supérieure à 20% du prix unitaire. »

```
1 // Solution dans VenteService::preparerLigneArticle()
2 $remiseMax = $produit->getPrixVente() * 0.20;
3 if ($remise > $remiseMax) {
4     throw new InvalidArgumentException(
5         "La remise dépasse le maximum autorisé de 20% (" . number_format(
6             $remiseMax, 0) . " FCFA)."
7     );
8 }
```

9.2.3 Scénario 3 : Blocage Automatique d'un Client Ayant des Dettes en Retard

Demande du formateur : « Interdisez tout nouvel achat à crédit si le client a au moins une dette avec statut *EN_RETARD*. »

```
1 // Solution dans ClientRepository ou VenteService
2 $stmtCheck = $pdo->prepare("SELECT COUNT(*) FROM dettes WHERE client_id = :
   cid AND statut_id = 3");
3 $stmtCheck->execute([':cid' => $clientId]);
4 if ((int)$stmtCheck->fetchColumn() > 0) {
5     throw new RuntimeException("Client bloqué : presence de creances
   impayees en retard.");
6 }
```

9.2.4 Scénario 4 : Ajout du Moyen de Paiement « Chèque »

Demande du formateur : « Intégrez le paiement par chèque dans le système. »

```
1 -- Insertion en base de donnees immediate sans modification de classe !
2 INSERT INTO modes_paiement (code, libelle, est_actif) VALUES ('CHEQUE', '
   Paiement par Ch que', TRUE);
```

9.2.5 Scénario 5 : Calcul du Taux de Rotation du Stock

Demande du formateur : « Ajoutez une méthode de valorisation globale du stock dans *ProduitRepository*. »

```
1 public function getValeurTotaleStock(): float
2 {
3     $stmt = $this->pdo->query("SELECT SUM(prix_achat * qte_stock) AS total
   FROM produits");
4     return (float)$stmt->fetchColumn();
5 }
```

9.2.6 Scénarios 6 à 10 : Réponses Rapides

- **Scénario 6 (Alerte Stock Email) :** Dans `validerVente()`, si `$produit->estEnAlerte()`, journaliser l'alerte critique dans un fichier de log.
- **Scénario 7 (Export CSV Journalier) :** Parcourir `$venteService->getVentesDuJour()` et formater les lignes avec `fputcsv($handle, ...)`.
- **Scénario 8 (Clôture de Caisse) :** Calculer la somme des encaissements par `mode_paiement_id` via `GROUP BY`.
- **Scénario 9 (Historique Client) :** Appeler `$detteRepository->findByClient($clientId)`.
- **Scénario 10 (Duplication de BL) :** Récupérer un Approvisionnement existant et réinjecter ses lignes dans `creerApprovisionnement()`.

9.3 15 Questions Pièges Fréquentes & Réponses Modèles pour l'Oral

Conseil Soutenance Orale & Questions Pièges : 1. Pourquoi avoir créé un Singleton pour Database ?

Réponse : Pour garantir qu'une seule et unique connexion PDO physique est ouverte par requête HTTP, économisant ainsi les sockets réseau et la mémoire vive du serveur SGBD.

Conseil Soutenance Orale & Questions Pièges : 2. Que fait PRAGMA foreign_keys = ON sous SQLite ?

Réponse : Par défaut, SQLite n'applique pas les contraintes d'intégrité référentielle. Cette directive active la vérification obligatoire des clés étrangères et des règles ON DELETE RESTRICT/CASCADE.

Conseil Soutenance Orale & Questions Pièges : 3. Pourquoi désactiver l'émulation des requêtes préparées (EMULATE_PREPARES => false) ?

Réponse : Pour forcer le SGBD à compiler la requête et à dissocier la syntaxe SQL des valeurs, garantissant une protection absolue contre toute forme d'injection SQL.

Conseil Soutenance Orale & Questions Pièges : 4. Que se passe-t-il si un crash survient pendant l'insertion des lignes de vente ?

Réponse : Le bloc catch (Throwable \$e) intercepte l'erreur et déclenche immédiatement \$pdo->rollBack(), annulant l'insertion de l'en-tête de vente et restaurant le stock initial.

Conseil Soutenance Orale & Questions Pièges : 5. Pourquoi avoir utilisé des classes pures plutôt que des enum ?

Réponse : Pour refléter fidèlement le schéma relationnel en 3FN (avec clés primaires id, codes et libellés) et permettre l'ajout dynamique de statuts ou moyens de paiement en SQL sans recompiler le code PHP.

Conseil Soutenance Orale & Questions Pièges : 6. Comment fonctionne la méthode getCreditDisponible() dans la classe Client ?

Réponse : Elle calcule la différence entre la limite de crédit autorisée et le total des dettes actuelles du client : max(0, limiteCredit - totalDettesActuelles). L'utilisation de max(0, ...) empêche tout résultat négatif aberrant si l'encours dépassait exceptionnellement la limite.

Conseil Soutenance Orale & Questions Pièges : 7. Pourquoi utiliser `password_hash()` et `password_verify()` avec BCrypt ?

Réponse : Bcrypt intègre automatiquement un sel aléatoire (*salt*) généré cryptographiquement et un facteur de coût (*cost factor*) qui ralentit les attaques par force brute et dictionnaire, rendant les tables arc-en-ciel (*rainbow tables*) inopérantes.

Conseil Soutenance Orale & Questions Pièges : 8. Quel est le rôle de la méthode `SessionManager : :regenerateId(true)` ?

Réponse : Elle régénère l'identifiant de cookie de session PHP immédiatement après l'authentification et supprime l'ancien identifiant, protégeant ainsi l'utilisateur contre les attaques de fixation de session (*Session Fixation*).

Conseil Soutenance Orale & Questions Pièges : 9. Pourquoi avoir séparé `VenteService` et `POSController` ?

Réponse : Pour respecter le principe de responsabilité unique (SRP). Le contrôleur gère uniquement la couche transport HTTP (requêtes, formulaires, redirections PRG, formatage JSON), tandis que le service encapsule la logique métier pure et les transactions SQL (calculs, déstockage, dettes).

Conseil Soutenance Orale & Questions Pièges : 10. Comment SQLite garantit-il la cohérence des transactions sans serveur dédié ?

Réponse : SQLite utilise un journal de rollback (*Rollback Journal*) ou un fichier WAL (*Write-Ahead Logging*) au niveau du système de fichiers de l'OS avec verrouillage exclusif du fichier `erp.db` pendant les blocs `BEGIN TRANSACTION` / `COMMIT`.

Conclusion & Perspectives

L'application **StoreManager Pro** démontre avec succès la viabilité et l'élégance d'une architecture **PHP 8.2+ POO Pure From Scratch** appliquée à la gestion commerciale intégrée. En s'affranchissant des dépendances lourdes des frameworks tout en respectant scrupuleusement les standards de génie logiciel (SOLID, MVC, Design Patterns, Transactions ACID, Résilience BDD), ce projet allie performance brute, robustesse financière et maintenabilité à long terme.

Perspectives d'Évolution Future

1. **API RESTful Sécurisée par JWT** : Exposition des points de terminaison pour connecter des terminaux mobiles de caisse sous Android/iOS.
2. **Synchronisation Multi-Magasins en Temps Réel** : Réplication asynchrone des stocks entre plusieurs succursales via WebSockets.
3. **Module d'Inventaire par Reconnaissance Visuelle** : Scan automatisé des rayonnages par caméra embarquée.

Annexes : Index des Classes & Tables SQL

Espace de Noms / Classe	Rôle dans l'Architecture
App\Core\Database	Singleton de gestion de la connexion PDO avec Fallback SQLite.
App\Core Router	Moteur de routage HTTP basé sur des expressions régulières.
App\Core\SessionManager	Gestion sécurisée des sessions, cookies et messages Flash.
App\Model\Entity\Produit	Entité métier : calcul des marges, seuils d'alerte et gestion du stock.
App\Model\Entity\Client	Entité métier : contrôle de solvabilité et plafond de crédit.
App\Model\Entity\Vente	Entité maître de facturation et encaissement en caisse.
App\Model\Entity\LigneVente	Ligne de détail d'une facture avec remises unitaires.
App\Model\Entity\Dette	Créance client avec échéance et historique de règlements.
App\Model\Entity\Paie ment	Versement ou acompte rattaché à une créance.
App\Model\Entity\Approvisionnement	Bon de livraison fournisseur et réceptions logistiques.
App\Model\Entity\LigneApprovisionnement	Ligne d'approvisionnement avec coût d'acquisition.
App\Model\Entity\User	Compte d'accès avec mot de passe haché et rôle RBAC.
App\Model\Entity\Role	Classe pure de référence pour les prérogatives d'accès.
App\Model\Entity\ModePaie ment	Classe pure de référence pour les canaux de règlement.
App\Model\Entity\StatutDette	Classe pure de référence pour les états de créance.
App\Model\Entity\StatutAppro	Classe pure de référence pour les états de bon de livraison.
App\Model\Entity\Categorie	Classe pure de référence pour les familles de produits.
App\Model\Entity\Fournisseur	Tiers fournisseur pour la chaîne d'approvisionnement.
App\Model\Repository*	9 classes d'accès aux données sécurisées par requêtes préparées.
App\Service\VenteService	Orchestration transactionnelle des ventes en caisse et des dettes.
App\Service\DebtService	Gestion des créances, recouvrements et amortissements.
App\Service\SupplyService	Gestion des réceptions de marchandises et des stocks.
App\Service\AuthManager	Gestion des habilitations RBAC et de la sécurité des accès.
App\Controller\POSController	Contrôleur de caisse tactile et gestion du panier.
App\Controller\AuthController	Contrôleur d'authentification et gestion des sessions.
App\Controller\DetteController	Contrôleur de gestion des créances et recouvrements.
App\Controller\SupplyController	Contrôleur des réceptions de bons de livraison.

TABLE 9.1 – Index des Composants Applicatifs de StoreManager Pro